Biometric-Bound Zero-Knowledge Voting: A Comparative Analysis of Face and Iris Identity in an End-to-End Verifiable Decentralized Architecture

Mahin R. Chowdhury*, Arpita Das*, K. M. Azharul Hasan*
*Department of Computer Science and Engineering,
Khulna University of Engineering & Technology,
Khulna-9203, Bangladesh

***Abstract*—Traditional end-to-end verifiable (E2EV) voting protocols usually depend on centralized credential servers for voter authentication, requiring trusted intermediaries that their cryptographic foundations seek to eliminate. We present BBZKV, a decentralized E2EV voting protocol that integrates identity verification directly within the cryptographic trust boundary by binding each voter to Poseidon-hashed Merkle commitments derived from two independent biometric modalities: facial and iris recognition. Built on a Hyperledger Fabric permissioned blockchain, the protocol stores only cryptographic commitments on-chain, ensuring that no plaintext biometric information is ever disclosed. During registration, biometric templates are committed to separate Poseidon-based Merkle trees. A single Groth16 zk-SNARK is used during the voting phase to simultaneously verify biometric authenticity, membership in the Merkle tree, and uniqueness of the election-specific nullifier, generating a fixed-size proof (about 192 bytes), verifiable in less than 40 ms on-chain. The protocol is formally proven secure within a $(\delta, \mathsf{FAR}_\delta)$-approximate Universal Composability (UC) framework, explicitly bounding cryptographic error by $\mathsf{negl}(\lambda)$ and residual identity error by the empirical false accept rate $\mathsf{FAR}_{256} = 0.047\%$. The dangers of off-chain verification are demonstrated through a proof-forgery attack targeting an API gateway. Vote privacy is preserved through exponential ElGamal homomorphic encryption, while ballot correctness is enforced using a disjunctive Chaum–Pedersen proof over the complete candidate ciphertext vector, preventing ballot-stuffing attacks. Double voting is eliminated through a Poseidon-derived, election-scoped nullifier recorded on the ledger. We further provide a circuit-level comparison of the two biometric verification mechanisms within a unified Groth16 framework. For the biometric sub-circuits, iris verification (256-bit Hamming distance) requires 8,224 R1CS constraints and 1.37 s WASM proof generation, whereas facial verification (64-dimensional squared cosine similarity) requires 9,484 constraints and 1.68 s (a 13.3% sub-circuit reduction). Across the full electoral proving circuit (including depth-20 Merkle tree membership and nullifier sub-circuits), total constraints are 19,724 (iris) versus 20,984 (face), achieving 163 ms versus 186 ms proving times under native C++ acceleration. This analysis reveals a formally characterized linear-versus-superlinear computational gap arising from the underlying arithmetic circuit structures. Collectively, BBZKV demonstrates that privacy-preserving biometric authentication, decentralized voter eligibility verification, and efficient end-to-end verifiability can be achieved within a single blockchain-based electoral framework.**

***Index Terms*—Zero-knowledge proofs, biometric authentication, homomorphic encryption, Merkle tree, blockchain, e-voting, Universal Composability, Groth16, Hyperledger Fabric.**

I. Introduction

To achieve E2EV e-voting, any protocol must ensure soundness regarding eligibility, ballot confidentiality, and universal verifiability even when malicious adversaries participate. Traditional solutions like Helios [**?**] and Belenios [**?**], while providing secure counting using homomorphic cryptography, rely on a centralized authority for registration and credential delivery, reintroducing a single point of failure: a compromised registry could silently issue fake credentials, inject phantom votes, and block valid participants with total disregard for the security provided by the cryptographic ledger. Consequently, trust must be established directly within the cryptographic boundaries of protocols [**?**], [**?**]. While biometrics (e.g., facial and iris features) offer a reliable solution on an individual basis without being transferable or replicable, and therefore can replace centralized credentials, transmitting raw biometric features over a public ledger threatens individual privacy, and any matching system requires transmitting sensitive data over the network for validation. Zk-SNARKs can help here – they prove to a third party (verifier) that a prover successfully executed some operations with a piece of data without directly revealing this specific piece or value as the output itself [**?**]. For example, World ID [**?**] uses iris-recognition-related zk proofs, but it focuses on proof of personhood and lacks any e-voting functionalities such as comparison-based votes or homomorphic tallying. The next logical question is then: which modality works more effectively in ZK settings, and what are the effective tradeoffs in terms of latency and security? First, the iris-based solution bases its evaluation on Hamming distance. Hamming distance fits linearly into an R1CS arithmetic circuit and grows linearly, whereas facial embedding matching is based on cosine similarity. The cosine calculation, involving the use of dot products, norm calculations, and comparators, can only fit non-linearly into the arithmetic circuit, thus producing a "super-linear" growth of constraint complexity. No prior work has estimated this arithmetic difference within a shared proving system, let alone combined multimodal biometric authentication with a proper security proof for E2EV e-voting systems. Moreover, the practical issues of security are often overlooked. For instance, if proof verification takes place at the API level in order to offload the blockchain, it creates a critical trust-boundary issue. Attackers

can directly bypass those verifications using an exploit on the API layer and continue the process. We describe BBZKV – a decentralized E2EV voting protocol. Each registered user's biometric is committed to a private Poseidon Merkle tree (no raw biometrics are stored on-chain). During voting, a single Groth16 zk-SNARK validates a user's biometric match against the registered commitment, membership in the Merkle tree, and that they have not already voted. Biometric and voting information are encrypted with exponential ElGamal homomorphic encryption, and an auditor can verify that the set of submitted votes is valid using a disjunctive Chaum–Pedersen proof on the full list of per-candidate encryptions. A novel election-scoped nullifier derived from Poseidon prevents double-voting. We provide a full UC security argument for BBZKV, demonstrating that it realizes ideal registration, voting, and tally functionalities under standard cryptographic assumptions. Our work is structured around three primary research questions:

- **RQ1 (Circuit Complexity):** How does the biometric modality (iris Hamming distance vs. Cosine similarity for face) affect R1CS constraint count, proof generation speed, and prover key size in an equivalent Groth16 electoral circuit?
- **RQ2 (Pipeline Latency and Scaling):** Under end-to-end prototyping of the entire cryptographic pipeline, what are the primary bottleneck phases, and how efficiently does the pipeline perform with populations up to 10^5 voters?
- **RQ3 (Trust Boundaries):** To what degree are different properties protected on the blockchain versus the API layer; does the vulnerability of API-side proof verification to theft remain exploitable, and if so, how? The effectiveness is demonstrated via concrete attack on prototype validation.

We make the following key contributions:

1) **Protocol Design Integration:** Construction of a composite E2EV e-voting protocol integrating two independent biometrics committed to private Poseidon Merkle trees, including nullifier-based anti double-voting, and ElGamal homomorphic voting.
2) **UC Security Proofs:** Rigorous formal security proof using the UC framework, achieving registration, voting, and tally ideal functionalities with assumptions q-SDH, collision-resistant/injective Poseidon hash functions, and DDH.
3) **Comparative Circuit Analysis:** Direct measurement and comparison within the same electoral proving environment of Hamming distance and cosine similarity, demonstrating linear versus superlinear complexity.
4) **Exploit Demonstration Evaluation:** Realization of the protocol in an example protocol; assessment of practical performance including between biometric modalities; illustration of necessary blockchain validation using a zero transaction fee exploit yielding perfect success rate.

The remainder of this paper is organized as follows. Section II surveys related work. Section III formalises the system and threat models. Section IV presents the complete protocol construction. Section V provides the formal security analysis. Section VI describes the implementation and evaluation, answering RQ1–RQ3. Section VII discusses limitations. Section VIII concludes.

II. Related Work

The literature relevant to BBZKV spans decentralized E2EV voting, privacy-preserving biometric authentication, and arithmetic circuit complexity. We survey these areas and identify key gaps below.

A. Decentralized End-to-End Verifiable Voting

Basic E2EV protocols like Helios [**?**] and Belenios [**?**] utilize homomorphic encryption to allow voters to verify their ballots. However, their voter authentication depends on a central credential server. A malicious registrar may issue dummy credentials or revoke real voter accounts and observers will not be able to detect these fraudulent activities. To provide good credential security and coercion resistant services, the concept of fake credentials, introduced by Juels et al. (JCJ) [**?**], was implemented in Civitas [**?**] with a quadratic tallying complexity. Current versions are designed to achieve linear complexity for coercion-resistant features [**?**], [**?**]. Moreover, blockchain-based protocols have transferred tallying and bulletin boards onto the blockchain [**?**], [**?**]. Semaphore [**?**] and MACI [**?**] utilize Merkle tree commitments and nullifiers to detect double voting, yet MACI is restricted to a single operator. SmartphoneDemocracy [**?**] adopts zk-SNARK for ballot and threshold decryption, and blind signatures on Fabric is shown to offer privacy and verifiability [**?**]. All these protocols assume prerequisite credentials and do not provide facilities for biometric enrollments to be within the scope of cryptographic trust.

B. Privacy-Preserving Biometric Authentication

Fuzzy extractors [**?**] and fuzzy commitments [**?**] are fuzzy biometric to key extraction mechanisms that depend on accurate characterization of the noise on biometrics. Matching can be performed on-chip given a certain threshold on similarity. Mao et al. [**?**] presented Euclidean distance matching within zk-SNARKs, and Guo et al. [**?**] presented combining commitments with SVMs.

Blanton et al. [**?**] tackled fingerprint matching using garbled circuits.

BioZero [**?**] and Zhang et al. [**?**] constructed a multibiometric committed identity on ledger, and ZK-SERIES [**?**] focused on temporal biometrics commitment verification. World ID [**?**] is the first system to commit Poseidon [**?**] to Semaphore [**?**] Merkle tree based Iris hash for proving World ID belongingness. Unfortunately, none of them support E2E voter private *and* homomorphic tallying for voting because they have no specific support for nullifiers. Besides, they use centralized servers or organizations.

For iris matching systems, utilizing FHE alone incurs a $120,000\times$ computational overhead compared with plain FHE [**?**]; this has been confirmed by studies on FHE for biometrics [**?**], [**?**], so circuits are significantly shortened [**?**].

C. Circuit Complexity in Biometric Proof Systems

The comparison function determines the constraint counts for the R1CS. Daugman's IrisCode [**?**], [**?**] operates over

TABLE I: Feature comparison of proposed architecture vs. existing literature. ✓ = fully addressed; ∼ = partially addressed; ✗ = not addressed.

Scheme	Decent. Auth.	ZK Bio.	HE Tally	Nullifier	E2EV	Biom. Modal.	UC Proof
Helios [?]	✗	✗	✓	✗	✓	None	✗
Belenios [?]	✗	✗	✓	✗	✓	None	✗
Civitas [?]	✗	✗	✗	∼	✓	None	✗
Semaphore [?]	✓	✗	✗	✓	✗	None	✗
MACI [?]	∼	✗	✓	✓	✓	None	✗
SmartphoneDem. [?]	∼	✗	✓	✓	✓	None	✗
World ID [?]	∼	✓	✗	✓	✗	Iris	✗
Guo et al. [?]	✓	✓	✗	✗	✗	Face	✗
BioZero [?]	✓	✓	✗	✗	✗	Multi	✗
Zhang et al. [?]	✗	✓	✗	✗	✗	Multi	✗
Wang et al. [?]	∼	✗	✓	✓	✓	None	✗
Huber & K. [?]	✗	✗	✓	✗	✓	None	✗
Proposed	✓	✓	✓	✓	✓	**Iris+Face**	✓

binary phase-sequence Hamming distance and may be implemented using circuits composed of one multiplication ($\mathsf{xor} j = aj + bj - 2ajbj$) and one additional constraint ($aj(1-a_j) = 0$) per code bit, resulting in a growth quadratic to length. Facial embeddings require calculation of the squared cosine similarity, involving the calculation of two inner products, two norms and an integer comparison circuit, resulting in a superlinear number of constraints. Overview comparisons exist comparing SNARKs, STARKs, and Bulletproofs [?], [?], however none specifically assess the trade-offs in circuit complexity between facial vs. Iris recognition.

D. Gap Analysis and Positioning

Table I summarizes the feature coverage of representative systems across the five properties we identify as jointly necessary for a secure electoral protocol: *decentralized biometric authentication*, *zero-knowledge template privacy*, *homomorphic tallying*, *nullifier-based anti-replay*, and *end-to-end verifiability*.

The table reveals a clean partition in the literature. Helios, Belenios, Civitas, MACI, SmartphoneDemocracy, and Wang et al. achieve E2EV tallying but rely on external credentials (biometrics are outside the trust boundary). World ID, Guo et al., BioZero, and Zhang et al. protect template privacy under zero-knowledge but lack nullifiers, tallying, or UC security proofs.

Three further gaps distinguish our contribution: (i) World ID relies on centralized Orb hardware and lacks e-voting; (ii) SmartphoneDemocracy uses eID credentials instead of biometric binding; and (iii) no prior work has quantified the face vs. iris circuit cost inside the same electoral proving stack. Our work addresses all three gaps.

III. System Model, Threat Model, and Security Definitions

This section formalizes the system architecture, the adversarial model, the security properties the protocol is designed to achieve, and the cryptographic building blocks upon which those guarantees rest.

A. System Entities

The protocol involves four classes of entities. We describe their roles, the secrets they hold, and the trust assumptions placed on each.

a) Voters ($\mathcal{V}$): Each voter, $\mathcal{V}i$, is a natural person intending to cast a single vote in election $\mathcal{E}$. Prior to registration, $\mathcal{V}i$ possesses an NID, a user password pwd$i$, and a live biometric sample. Upon registration, $\mathcal{V}i$ obtains a secret key $ski \in \mathbb{F}p$ and an encrypted QR credential containing the salt s_i, NID hash, and the biometric configuration. Voters are assumed to be rational but potentially malicious (e.g., attempting double-voting or ballot stuffing), yet they would never voluntarily share their biometrics (Assumption III.1).

b) Registration Authority ($\mathcal{RA}$): The $\mathcal{RA}$ supports supervised registration. The $\mathcal{RA}$ verifies the validity of NIDs, acquires biometrics, extracts biometric templates, generates keypairs, submits commitments $Ci = \mathcal{H}2(hi, ski)$ on-chain and distributes encrypted QR credentials. The $\mathcal{RA}$ is assumed honest for identity verification on enrollment and should not admit phantom voters. It remains offline during voting and cannot decrypt ballot content nor proofs.

c) Tallying Authority ($\mathcal{TA}$): The $\mathcal{TA}$ generates ElGamal keys $(x, h = xG)$, homomorphically aggregates votes following the election close, decrypts the aggregate sum using the BSGS algorithm, and publishes the results together with a proof of decryption. The $\mathcal{TA}$ is assumed not to decrypt single ballots ahead of vote aggregation (addressed via thresholding) and does not control voter eligibility.

d) Blockchain Ledger ($\mathcal{L}$): We use Hyperledger Fabric as a permissioned ledger. It stores commitments, the nullifier set, and the encrypted ballots. We rely on its append-only guarantee, ordering guarantee of transactions within blocks, and write validation. We do not expect it to ensure template privacy or ballot content confidentiality.

B. Threat Model and Trust Assumptions

We consider a probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ who may corrupt a subset of system entities. We define four adversarial classes, summarized in Table II along with their respective attack vectors and mitigations:

- **Malicious Voter ($\mathcal{A}_{\mathcal{V}}$):** Controls credentials and biometric data to attempt double voting, impersonation, credential sharing, or ballot stuffing.
- **Malicious Registration Authority ($\mathcal{A}_{\mathcal{RA}}$):** Controls enrollment to register phantom voters, steal templates, or harvest keys.
- **Malicious Tallying Authority ($\mathcal{A}_{\mathcal{TA}}$):** Controls tallying keys to decrypt individual ballots, falsify tallies, or exclude votes.
- **Passive Ledger Observer ($\mathcal{A}_{\mathcal{L}}$):** Observes commitments $\{C_1, \ldots, C_N\}$, nullifiers $\{\nu_1, \ldots, \nu_M\}$, SNARK proofs $\{\Pi_1, \ldots, \Pi_M\}$, and encrypted votes.

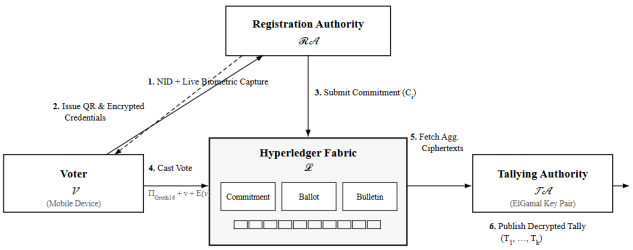


Fig. 1: High-level system architecture demonstrating the remote e-voting lifecycle.

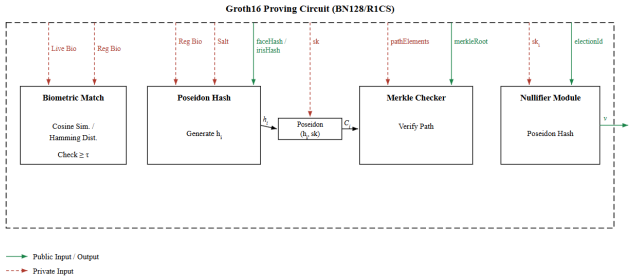


Fig. 2: Modular architecture of the zk-SNARK authentication circuit.

TABLE II: Threat Matrix and BBZKV Mitigations

Adversary	Attack Vector	BBZKV Mitigation
Malicious Voter ($\mathcal{A}_{\mathcal{V}}$)	Double voting	Nullifier $\nu = \mathcal{H}_2(sk, \text{electionId})$ is deterministic; chaincode rejects duplicates.
	Impersonation	zk-SNARK circuit enforces biometric similarity; without genuine trait, proof fails.
	Ballot stuffing	Disjunctive Chaum–Pedersen proof constrains ciphertext to exactly 0 or 1.
	Credential sharing	Proxy must also present matching live biometric; without it, SNARK fails.
Malicious RA ($\mathcal{A}_{\mathcal{RA}}$)	Phantom voter	Mitigated by NID deduplication check and offline census audits.
	Template theft	Templates are hashed immediately via Poseidon. Recovering plaintext requires inverting Poseidon.
	Secret key theft	sk_i is encrypted via PBKDF2-AES-256-CBC; $\mathcal{RA}$ does not retain the password.
Malicious TA ($\mathcal{A}_{\mathcal{TA}}$)	Individual decryption	Mitigated via t-of-n threshold key sharing. Anonymity remains protected by SNARKs.
	Tally falsification	Universal verifiability: observers can re-aggregate on-chain ciphertexts.
	Selective exclusion	Observers check completeness against the on-chain `VoteCast` event log.

1) Collusion Scenarios:

- $\mathcal{A}_{\mathcal{RA}} + \mathcal{A}_{\mathcal{L}}$**:** Without sk_i, the $\mathcal{RA}$ cannot link known commitments C_i to observed nullifiers ν_j.
- $\mathcal{A}_{\mathcal{RA}} + \mathcal{A}_{\mathcal{TA}}$**:** Neither party can link C_i to a specific encrypted ballot without sk_i, preserving ballot secrecy.
- $\mathcal{A}_{\mathcal{TA}} + \mathcal{A}_{\mathcal{L}}$**:** The $\mathcal{TA}$ can decrypt ballots and observe nullifiers, but cannot link nullifiers to voter identities without sk_i.

C. Security Definitions

Assumption III.1 (Uncloneable Biometrics)**.** *An adversary $\mathcal{A}$ cannot extract or replicate the physical biometric template of an honest voter $\mathcal{V}_i$ to pass the matching threshold, nor can a voter voluntarily surrender their biometric trait to an impersonator without physical presence.*

Definition 1 (Eligibility Verifiability). An e-voting protocol satisfies *eligibility verifiability* if for every PPT adversary $\mathcal{A}$:

$$\Pr\left[\mathcal{A} \text{ produces valid } (\Pi, \nu, E(\mathbf{v})) \text{ accepted by chaincode} \mid \mathcal{A} \notin \mathcal{V}_{\text{registered}}\right] \leq \text{negl}(\lambda) \tag{1}$$

Intuition. To forge a proof, $\mathcal{A}$ must break Poseidon preimage resistance, find a collision, or break Groth16 soundness. Furthermore, $\mathcal{A}$ must present a live biometric sample satisfying the in-circuit similarity check.

Definition 2 (Ballot Secrecy). An e-voting protocol satisfies *ballot secrecy* if for every PPT adversary $\mathcal{A}$:

$$\left|\Pr[\mathcal{A} \text{ identifies } v_i \mid \text{view}(\mathcal{A})] - \frac{1}{k}\right| \leq \text{negl}(\lambda) \tag{2}$$

Intuition. Rests on the IND-CPA security of exponential ElGamal under the DDH assumption, and the zero-knowledge property of Groth16.

Definition 3 (Anti-Replay / Double-Vote Prevention). An e-voting protocol satisfies *anti-replay* if for every PPT adversary $\mathcal{A}$:

$$\Pr[\mathcal{A} \text{ casts two accepted votes in } \mathcal{E}] \leq \text{negl}(\lambda) \tag{3}$$

Intuition. The nullifier $\nu = \mathcal{H}_2(sk, \text{electionId})$ is deterministic and recorded on-chain. Finding a collision is negligible under Poseidon's collision resistance.

Definition 4 (Universal Verifiability). Any observer $\mathcal{O}$ without secret keys can verify: (i) individual SNARK proof validity, (ii) vote well-formedness via Chaum-Pedersen proofs, (iii) tally correctness via homomorphic aggregation, (iv) nullifier uniqueness, and (v) completeness against the event log.

Definition 5 (Voter Anonymity / Unlinkability). For every PPT adversary $\mathcal{A}$ with ledger read access:

$$\Pr\left[\mathcal{A} \text{ links } (\Pi_j, \nu_j, E(\mathbf{v}_j)) \text{ to } \mathcal{V}_i\right] \leq \frac{1}{|\mathcal{V}_{\text{registered}}|} + \text{negl}(\lambda) \tag{4}$$

D. Cryptographic Preliminaries

1) Poseidon Hash Function: Poseidon [**?**] is an arithmetization-friendly hash function over $\mathbb{F}_p$. We use Poseidon_8, Poseidon_4, and Poseidon_2 for commitments, nullifiers, and Merkle nodes. Security holds under the algebraic group model (AGM). Poseidon requires $\sim$300 R1CS constraints for arity 2, compared to $\sim$25,000 for SHA-256.

2) Groth16 zk-SNARK: Groth16 [**?**] provides knowledge soundness under the q-SDH assumption on BN128 and perfect zero-knowledge. Proofs are constant size ($\sim$192 bytes) and require a trusted setup ceremony (Powers of Tau).

3) Exponential ElGamal Encryption: Given public key $h = xG$ and plaintext $m \in \{0, 1\}$:

$$E(m; r) = (c_1, c_2) = (rG, mG + rh), \quad r \xleftarrow{\$} \mathbb{Z}_n^* \tag{5}$$

Additive homomorphism enables tallying:

$$\begin{aligned} &E(m_1; r_1) \oplus E(m_2; r_2) \\ &= \big((r_1 + r_2)G, (m_1 + m_2)G + (r_1 + r_2)h\big) \\ &= E(m_1 + m_2; r_1 + r_2) \end{aligned} \tag{6}$$

Security rests on the DDH assumption over secp256k1.

4) Disjunctive Chaum–Pedersen Proof: A non-interactive Σ-OR proof [**?**] demonstrating $m \in \{0, 1\}$. Security relies on the discrete logarithm assumption and the Fiat–Shamir heuristic.

5) Merkle Tree (Poseidon-Based): A binary tree of depth $L = 20$. Membership proofs provide hiding (preserving anonymity) and binding (under Poseidon collision resistance).

6) Biometric Similarity Metrics: For cosine similarity (face channel) with integer-scaled vectors:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} \tag{7}$$

For Hamming distance (iris channel) over $n = 256$ bits, theoretical false-match probability is:

$$P_{\text{false}} = \sum_{k=0}^{d} \binom{n}{k} p^k (1 - p)^{n-k} \tag{8}$$

7) Key Derivation: PBKDF2 with HMAC-SHA-256 (100,000 iterations) derives a 256-bit AES key for symmetric credential encryption.

Our security architecture relies on standard cryptographic primitives: Groth16 zero-knowledge proofs under the q-SDH assumption over the BN128 scalar field (providing $\sim$128-bit security) with perfect zero-knowledge; the Poseidon hash function under algebraic hardness assumptions in the algebraic group model (AGM) over BN128; ElGamal encryption and Chaum–Pedersen disjunctive proofs under the DDH and discrete logarithm assumptions over the secp256k1 curve (providing $\sim$128-bit security); and symmetric credential encryption via AES-256-CBC (providing 256-bit security).

IV. Protocol Construction

This section presents the complete protocol construction. We describe six phases: key generation, voter registration, authentication with zk-SNARK proof generation, ballot creation, vote casting, and homomorphic tallying. Throughout, we use the notation summarized in Table III.

A. Key Generation and Commitment Scheme

The protocol employs three distinct key generation phases: (i) SNARK-friendly voter credentials, (ii) per-election ballot encryption, and (iii) Hyperledger Fabric organizational keys.

TABLE III: Notation Summary

Symbol	Description
$\mathbb{F}_p$	BN128 scalar field, $p \approx 2.188 \times 10^{77}$
$\mathcal{H}$	Poseidon hash function over $\mathbb{F}_p$ [**?**]
$\mathcal{H}_2(a, b)$	Poseidon hash of two field elements
$\mathsf{SHA256}(\cdot)$	SHA-256 hash (used outside circuits only)
$\mathbf{e}_{\text{face}} \in \mathbb{R}^{64}$	L2-normalized PCA-reduced embedding
$\mathbf{c}_{\text{iris}} \in \{0, 1\}^{256}$	Downsampled binary iris code
s	Random registration salt
sk	Voter secret key (≤ 252 bits)
C	Voter commitment leaf
ν	Nullifier
$\mathcal{T}^{(\text{face})}, \mathcal{T}^{(\text{iris})}$	Independent Poseidon Merkle trees ($L = 20$)
G	Generator of secp256k1
$h = xG$	ElGamal public key
Π_{Groth16}	Groth16 zk-SNARK proof
σ	Scale factor (10^6) for integerization

1) Voter Credential Generation: During registration, each voter $\mathcal{V}_i$ is assigned a unique secret key sk_i drawn uniformly at random from $\{1, \ldots, 2^{252} - 1\} \subset \mathbb{F}_p$. The upper bound of 252 bits ensures that sk_i lies strictly within the BN128 scalar field, preventing modular reduction artifacts in the Poseidon circuit. Concretely:

$$sk_i \leftarrow \{0, 1\}^{256} \bmod 2^{252}, \quad sk_i \neq 0 \tag{9}$$

Given a biometric template $\mathbf{b}_i$ (either a facial embedding or an iris code) and a random salt $s_i \in \mathbb{F}_p$, the system computes:

$$h_i = \mathcal{H}_{\text{chunked}}(\mathbf{b}_i, s_i) \tag{10}$$

where $\mathcal{H}_{\text{chunked}}$ denotes a two-stage Poseidon hash. For the facial channel ($\mathbf{b}_i \in \mathbb{Z}^{64}$), the 64 scaled integer values are partitioned into 8 chunks of 8 elements each; each chunk is hashed with Poseidon_8, and the resulting 8 digests are hashed together via Poseidon_8, yielding a single field element h_{emb}. The final biometric hash is then:

$$h_i^{(\text{face})} = \mathcal{H}_2(h_{\text{emb}}, s_i) \tag{11}$$

For the iris channel ($\mathbf{b}_i \in \{0, 1\}^{256}$), the 256 bits are packed into 32 bytes via $\texttt{Bits2Num}(8)$, the 32 packed values are hashed in 4 chunks of 8 via $\text{Poseidon}_8 \rightarrow \text{Poseidon}_4$, and the result is salted identically:

$$h_i^{(\text{iris})} = \mathcal{H}_2(h_{\text{pack}}, s_i) \tag{12}$$

The voter commitment leaf is then:

$$C_i = \mathcal{H}_2(h_i, sk_i) \tag{13}$$

This two-layer construction serves dual purposes: the inner hash h_i binds the biometric template to a random salt (preventing dictionary attacks on known biometric distributions), and the outer commitment C_i binds the biometric hash to a secret key (enabling nullifier derivation without revealing h_i).

2) ElGamal Encryption Keypair: For each election $\mathcal{E}$, a tallying authority generates an exponential ElGamal keypair over the secp256k1 curve:

$$x \xleftarrow{\$} \mathbb{Z}_n^*, \quad h = xG \tag{14}$$

where G is the generator of secp256k1 and n is its order. The public key h is published on the Fabric ledger; the private key x is retained by the tallying authority.

3) Fabric Network Keys: The Hyperledger Fabric network operates with standard X.509 organizational certificates issued by a Fabric Certificate Authority. The voting smart contract (chaincode) runs on the `mychannel` channel with an endorsement policy requiring validation from authorized peers.

B. Voter Registration Phase

Registration is a one-time operation performed at a supervised enrollment station. The protocol proceeds as follows.

Step 1: Identity deduplication. The NID number is hashed: $\text{nidHash} = \mathsf{SHA256}(\text{NID})$. This is submitted to the Fabric chaincode to check for prior registration.

Step 2: Biometric feature extraction. Depending on the chosen channel:

- *Face channel.* The face image is processed via FaceNet [**?**] to obtain a 128-dimensional embedding. The embedding is L2-normalized, projected to 64 dimensions via a pre-trained PCA model, and re-normalized:

$$\begin{aligned} \mathbf{v} &= \frac{\text{FaceNet}(I)}{\|\text{FaceNet}(I)\|_2} \\ \hat{\mathbf{e}} &= \frac{\text{PCA}_{64}(\mathbf{v})}{\|\text{PCA}_{64}(\mathbf{v})\|_2} \in \mathbb{R}^{64} \end{aligned} \tag{15}$$

  The floating-point embedding is then scaled: $\mathbf{e}_j^{(\text{int})} = \lfloor \hat{e}_j \cdot \sigma + 0.5 \rfloor$, where $\sigma = 10^6$.
- *Iris channel.* The eye image undergoes normalization and noise mask generation. Three 2-D Log-Gabor filters convolve the strip, phase-quantized to 2 bits per pixel, and downsampled to 256 bits. This downsampling reduces the template size from the raw 196 K bits to a compact 256-bit code. In zero-knowledge arithmetic circuits, proving operations over 196 K bits would require over 6 million constraints, which exceeds the proving key and memory budgets of commodity client devices. Downsampling to 256 bits compresses the sub-circuit constraint budget to just 8,224 constraints, keeping witness and proof generation tractable for client-side execution.

Step 3: Cryptographic commitment. The biometric hash h_i, secret key sk_i, and commitment C_i are generated.

Step 4: On-chain registration. The commitment C_i is submitted to the Fabric chaincode. Face and iris registrations populate separate Merkle trees:

$$\mathcal{T}^{(\text{face})} = \text{SMT}(\{C_1^{(\text{face})}, \ldots, C_m^{(\text{face})}\}) \tag{16}$$

$$\mathcal{T}^{(\text{iris})} = \text{SMT}(\{C_1^{(\text{iris})}, \ldots, C_k^{(\text{iris})}\}) \tag{17}$$

Step 5: Credential issuance. SNARK credentials and a lightweight QR credential are encrypted via AES-256-CBC and PBKDF2, ensuring no plaintext data is exposed.

C. Authentication and zk-SNARK Proof Generation

Authentication produces a Groth16 [**?**] zero-knowledge proof Π attesting to biometric similarity, commitment integrity, Merkle-tree membership, and nullifier correctness.

1) Groth16 Proving System: A Groth16 proof for an R1CS consists of three group elements $\Pi = (\pi_A \in \mathbb{G}_1, \pi_B \in \mathbb{G}_2, \pi_C \in \mathbb{G}_1)$ over the BN128 curve. The Common Reference String (CRS) is generated via a two-phase trusted setup ceremony, yielding a Phase 1 Powers of Tau output $\{\tau^k G_1\}_{k=0}^{N-1}$ and $\{\tau^k G_2\}_{k=0}^{N-1}$, and a Phase 2 circuit-specific `.zkey` proving key.

2) Circuit Compilation Pipeline: Written in Circom 2.1.6 [**?**], the circuits compile into an R1CS constraint system $\mathbf{a}_i \cdot \mathbf{w} \times \mathbf{b}_i \cdot \mathbf{w} = \mathbf{c}_i \cdot \mathbf{w}$ alongside C++ and WASM witness generators.

3) Sub-Circuit Architecture: Each circuit is composed of four modular sub-circuits:

a) PoseidonHashChunked: Hashes the registered biometric template. For iris, an additional 512 binary enforcement constraints ($a_j(1 - a_j) = 0$) are applied.

b) MerkleTreeChecker: Verifies $C_i \in \mathcal{T}$ via a depth $L = 20$ path:

$$\text{current}_{\ell+1} = \text{Poseidon}_2(\text{left}, \text{right}) \tag{18}$$

c) SquaredCosineSimilarity: Proves face similarity without division/square roots:

$$\text{LHS} = \text{dot}^2 \cdot t_{\text{den}}, \quad \text{RHS} = t_{\text{num}} \cdot \|\mathbf{a}\|^2 \cdot \|\mathbf{b}\|^2 \tag{19}$$

The algebraic soundness of this constraint follows by squaring both sides of the target cosine similarity threshold assertion. Given vectors $\mathbf{a}, \mathbf{b}$ and similarity threshold $T = \sqrt{t_{\text{num}}/t_{\text{den}}}$, we seek to assert that $\cos(\mathbf{a}, \mathbf{b}) \geq T$. Since both sides are non-negative, this is equivalent to $\cos^2(\mathbf{a}, \mathbf{b}) \geq T^2$, which can be expanded to $(\mathbf{a} \cdot \mathbf{b})^2/(\|\mathbf{a}\|^2\|\mathbf{b}\|^2) \geq t_{\text{num}}/t_{\text{den}}$. Multiplying through by the denominators yields the equivalent polynomial relation $(\mathbf{a} \cdot \mathbf{b})^2 \cdot t_{\text{den}} \geq t_{\text{num}} \cdot \|\mathbf{a}\|^2\|\mathbf{b}\|^2$, which is computed entirely using integer field multiplications and comparisons, eliminating non-linear division and square-root operations from the R1CS system.

d) HammingDistance: Proves iris distance via arithmetic XOR ($\text{xor}_j = a_j + b_j - 2a_j b_j$) followed by summation.

e) Nullifier Sub-Circuit: Enforces $\nu = \mathcal{H}_2(sk_i, \text{electionId})$.

4) Witness Preparation and Merkle Tree Construction: A sparse Poseidon Merkle tree is constructed client-side. Zero hashes are precomputed ($z_0 = 0, z_{\ell+1} = \mathcal{H}_2(z_\ell, z_\ell)$), reducing tree construction complexity from $O(2^L)$ to $O(N \cdot L)$ evaluations.

5) Circuit Signal Summary: The circuit signals comprise public inputs (such as the biometric template hash, Merkle root, election-scoped nullifier, and matching thresholds) and private inputs (such as the live biometric template, registered biometric template, voter secret key, salt, and Merkle membership path). Table I in the Supplementary Material provides the detailed visibility, modality, and description of each signal.

6) Constraint Complexity Analysis: Table II in the Supplementary Material summarizes the constraint complexity and provides the detailed sub-circuit R1CS constraint breakdown, which includes Poseidon hashes, packing operations, binary checks, and threshold comparisons for both modalities.

D. Ballot Creation

The election identifier used in the nullifier is derived deterministically:

$$\text{electionId} = \text{trunc}_{128}\left(\mathsf{SHA256}(\mathcal{B}.\text{id})\right) \bmod p \tag{20}$$

E. Vote Casting

Step 1 & 2: Validation & Double-Vote Check. The chaincode checks $\text{hasVoted}(\nu) \stackrel{?}{=} \text{true}$.

Step 3 & 4: zk-SNARK & Encryption. For vote vector $\mathbf{v} \in \{0,1\}^k$, exponential ElGamal ciphertexts are computed:

$$E(v_j) = (c_{1,j}, c_{2,j}) = (r_j G, v_j G + r_j h) \tag{21}$$

Step 5: Disjunctive validity proof. A Σ-OR proof guarantees $v_j \in \{0,1\}$. The Fiat-Shamir challenge is:

$$\begin{aligned} e = \mathsf{SHA256}(&c_{1,j} \parallel c_{2,j} \parallel a_1 \parallel b_1 \\ &\parallel a_2 \parallel b_2) \bmod n \end{aligned} \tag{22}$$

Step 6: On-chain submission. The chaincode atomically verifies proofs, records the encrypted ballot, and stores the nullifier.

F. Homomorphic Tallying and Result Decryption

Step 1: Ciphertext aggregation.

$$\begin{aligned} \hat{E}(T_j) &= \sum_{i=1}^{N} E(v_{i,j}) \\ &= \left(\sum_{i=1}^{N} r_{i,j} G,\ T_j G + \sum_{i=1}^{N} r_{i,j} h \right) \end{aligned} \tag{23}$$

Step 2: Decryption.

$$M_j = \hat{c}_{2,j} - x \cdot \hat{c}_{1,j} = T_j G \tag{24}$$

Step 3: Discrete logarithm recovery. The aggregate T_j is solved via Baby-Step Giant-Step (BSGS) in $O(\sqrt{N})$ time.

The prototype implementation is structured as follows: registration and witness preparation are implemented in `api-server/server.js` and `crypto/snark.js` / `crypto/iris-snark.js`; commitment storage is handled by `identityContract.js`; zero-knowledge proof generation and validation are orchestrated via client-side scripts; and homomorphic aggregation and tallying are managed in `routes/tally.js` and `crypto/homomorphic.js`.

V. Security Analysis

This section presents the formal security analysis of BBZKV. We proceed in five stages: First, we define the Universal Composability (UC) framework setup, the $(\mathcal{F}_{\mathsf{CRS}}, \mathcal{F}_{\mathsf{ledger}})$-hybrid model, and the three ideal functionalities ($\mathcal{F}_{\mathsf{reg}}$, $\mathcal{F}_{\mathsf{vote}}$, $\mathcal{F}_{\mathsf{tally}}$) in §V-A. Second, we construct an explicit, polynomial-time Ideal-World Simulator $\mathcal{S}$ in §V-B. Third, we state and prove the Main UC Realization Theorem (§V-C) via a sequence of computationally indistinguishable hybrid environments ($\mathcal{H}_0 \ldots \mathcal{H}_4$), bounding the environment's distinguishing advantage strictly by the hardness of underlying cryptographic primitives. Fourth, we derive the five core security properties—Eligibility Soundness, Anti-Replay, Voter Anonymity, Ballot Secrecy, and Universal Verifiability—as formal corollaries of the main UC theorem and primitive hardness reductions (§V-E1–§V-E5). Fifth, we evaluate multi-entity collusion dynamics and detail deployment trust-boundary gaps (§V-F–§V-G).

Throughout, $\lambda \in \mathbb{N}$ denotes the security parameter; all entities and algorithms are probabilistic polynomial-time (PPT) unless explicitly noted. $\mathsf{negl}(\lambda)$ denotes a negligible function. We write $x \stackrel{\$}{\leftarrow} S$ for uniform sampling from set S, and $\mathsf{Adv}^{\mathsf{X}}_{\mathsf{Y}}(\mathcal{B})$ for the advantage of adversary $\mathcal{B}$ breaking primitive Y in security game X. The underlying cryptographic instantiation employs Groth16 over BN128, Poseidon hashes over $\mathbb{F}_p$, exponential ElGamal over secp256k1, and Hyperledger Fabric as the permissioned ledger. Merkle tree depth is fixed at $L = 20$.

a) Target Protocol vs. Prototype Implementation: We emphasize that the formal security model and UC realization theorems described in this section define the *target* protocol design, where all cryptographic verifications (Groth16 ZK-SNARK and Chaum–Pedersen proofs) are executed on-chain via smart contracts. The prototype implementation, evaluated in Section VI, uses an off-chain API-side optimization. This creates a trust-boundary gap between the formal protocol and the prototype, which we analyze and exploit in §V-G.

A. UC Ideal Functionalities

We formulate security in Canetti's Universal Composability (UC) framework [**?**]. A protocol π UC-realizes an ideal functionality $\mathcal{F}$ in the $(\mathcal{F}_{\mathsf{CRS}}, \mathcal{F}_{\mathsf{ledger}})$-hybrid model if for every PPT real-world adversary $\mathcal{A}$, there exists a PPT ideal-world simulator $\mathcal{S}$ such that no PPT environment $\mathcal{Z}$ can distinguish interactions with $(\pi, \mathcal{A})$ from interactions with $(\mathcal{F}, \mathcal{S})$:

$$\left|\Pr[\mathcal{Z}^{\pi,\mathcal{A}}(1^\lambda) = 1] - \Pr[\mathcal{Z}^{\mathcal{F},\mathcal{S}}(1^\lambda) = 1]\right| \le \mathsf{negl}(\lambda) \tag{25}$$

We define three joint ideal functionalities over party set $\mathcal{P} = \mathcal{V} \cup \{\mathcal{RA}, \mathcal{TA}, \mathcal{L}\}$.

a) Functionality $\mathcal{F}_{\mathsf{reg}}$ (Registration): On receiving $(\mathsf{Register}, nid_i, \mathbf{b}_i, mode)$ from voter $\mathcal{V}_i$ via $\mathcal{RA}$:

1) Abort if nid_i is already marked as registered.
2) Sample secret key $sk_i \stackrel{\$}{\leftarrow} \mathbb{F}_p^*$ and salt $s_i \stackrel{\$}{\leftarrow} \mathbb{F}_p$.
3) Compute $h_i \leftarrow H_{\mathsf{chunked}}(\mathbf{b}_i, s_i)$ and leaf commitment $C_i \leftarrow H_2(h_i, sk_i)$.
4) Record tuple $(i, nid_i, h_i, C_i, mode)$ and mark nid_i as registered.
5) Transmit C_i to $\mathcal{F}_{\mathsf{ledger}}$ for insertion into Merkle tree $\mathcal{T}^{(mode)}$, and return (C_i, sk_i, s_i) encrypted under pwd_i to $\mathcal{V}_i$.

Leakage Policy: Leaks only $(C_i, mode)$ to $\mathcal{S}/\mathcal{A}$. Plaintext tuple $(\mathbf{b}_i, sk_i, s_i, h_i)$ is never leaked.

b) Functionality $\mathcal{F}_{\mathsf{vote}}$ (Voting): On receiving $(\mathsf{Vote}, C_i, \nu_i, \mathbf{v}_i, \Pi_i, \{(c_{1,j}, c_{2,j}, \pi_j^{\mathsf{val}})\}_j)$ from voter $\mathcal{V}_i$:

1) Abort if leaf $C_i \notin \mathcal{T}$.
2) Abort if nullifier $\nu_i \in \mathcal{N}$ (double-vote check).
3) Abort if candidate vector $\mathbf{v}_i \notin \{0,1\}^k$ or $\sum_{j=1}^{k} v_{i,j} \neq 1$ (invalid ballot check).

4) Abort if Groth16 proof Π_i or disjunctive proofs $\{\pi_j^{\mathsf{val}}\}$ fail verification.
5) Append ν_i to set $\mathcal{N}$ and record ciphertexts $\{(c_{1,j}, c_{2,j})\}_j$ on $\mathcal{F}_{\mathsf{ledger}}$.

Leakage Policy: Leaks only $(\nu_i, \{(c_{1,j}, c_{2,j}, \pi_j^{\mathsf{val}})\}_j)$ to $\mathcal{S}/\mathcal{A}$. Plaintext choice $\mathbf{v}_i$, secret key sk_i, and linkage between C_i and ν_i are never leaked.

c) Functionality $\mathcal{F}_{\mathsf{tally}}$ (Tallying): On receiving Tally from $\mathcal{TA}$ after election closing:

1) Retrieve all committed ciphertext vectors from $\mathcal{F}_{\mathsf{ledger}}$.
2) Compute homomorphic product $\hat{C}_j = \bigoplus_{i=1}^{M} (c_{1,j}^{(i)}, c_{2,j}^{(i)})$ for candidate $j \in \{1, \ldots, k\}$.
3) Decrypt $\hat{C}_j$ using secret key x and extract election tally T_j via Baby-Step Giant-Step (BSGS) discrete logarithm solver.
4) Output tally vector $\{T_j\}_{j=1}^{k}$, aggregate ciphertexts $\{\hat{C}_j\}_{j=1}^{k}$, and Chaum–Pedersen decryption correctness proofs $\{\pi_j^{\mathsf{dec}}\}_{j=1}^{k}$ to all parties.

Leakage Policy: Leaks only total candidate counts $\{T_j\}_j$ and aggregate ciphertexts $\{\hat{C}_j\}_j$. Individual voter choices are never decrypted.

B. Explicit Construction of Ideal-World Simulator

To prove UC security, we construct an explicit PPT Ideal-World Simulator $\mathcal{S}$ interacting with environment $\mathcal{Z}$, real adversary $\mathcal{A}$, and ideal functionalities $\mathcal{F}_{\mathsf{BBZKV}} = (\mathcal{F}_{\mathsf{reg}}, \mathcal{F}_{\mathsf{vote}}, \mathcal{F}_{\mathsf{tally}})$. $\mathcal{S}$ simulates the CRS, ledger, and network communications as detailed below:

1) **Setup Phase:** $\mathcal{S}$ simulates $\mathcal{F}_{\mathsf{CRS}}$ by generating Groth16 keypair $(\mathsf{pk}, \mathsf{vk})$ along with simulation trapdoor $\tau = (\alpha, \beta, \gamma, \delta, x)$. $\mathcal{S}$ publishes $(\mathsf{pk}, \mathsf{vk})$ to $\mathcal{Z}$ and $\mathcal{A}$. $\mathcal{S}$ also initializes simulated ledger state $\mathcal{L}$ maintaining commitment trees $\mathcal{T}$ and nullifier set $\mathcal{N}$.
2) **Registration Phase (Honest Voter $\mathcal{V}_i$):** Upon receiving notification $(C_i, mode)$ from $\mathcal{F}_{\mathsf{reg}}$, $\mathcal{S}$ records commitment C_i on simulated ledger $\mathcal{L}$ and updates Merkle tree $\mathcal{T}^{(mode)}$.
3) **Registration Phase (Corrupt Voter $\mathcal{A}$):** When $\mathcal{A}$ submits $(\mathsf{Register}, nid_a, \mathbf{b}_a, mode)$ to the simulated registration station, $\mathcal{S}$ forwards $(\mathsf{Register}, nid_a, \mathbf{b}_a, mode)$ to $\mathcal{F}_{\mathsf{reg}}$ on $\mathcal{A}$'s behalf and updates $\mathcal{T}^{(mode)}$.
4) **Voting Phase (Honest Voter $\mathcal{V}_i$):** When $\mathcal{F}_{\mathsf{vote}}$ sends leakage $(\nu_i, \{(c_{1,j}, c_{2,j})\}_j)$, $\mathcal{S}$ does not know voter choice $\mathbf{v}_i$. $\mathcal{S}$ creates simulated vote encryptions $E(\mathbf{0}) = (r_{i,j}G, r_{i,j}h)$ for dummy zero vectors. $\mathcal{S}$ then uses Groth16 simulation trapdoor τ to generate simulated zero-knowledge proof $\Pi_i \leftarrow \mathsf{Sim}_{\mathsf{Groth16}}(\mathsf{vk}, \tau, x_i)$ over public statement $x_i = (R, \nu_i, \mathrm{electionId}, \ldots)$ without a witness. $\mathcal{S}$ simulates disjunctive Chaum–Pedersen proofs π_j^{val} using the standard Σ-OR zero-knowledge simulator. $\mathcal{S}$ records $(\nu_i, \Pi_i, \{c_{1,j}, c_{2,j}, \pi_j^{\mathsf{val}}\}_j)$ on simulated ledger $\mathcal{L}$.
5) **Voting Phase (Corrupt Voter $\mathcal{A}$):** When $\mathcal{A}$ posts tuple $(\Pi^*, \nu^*, \{(c_{1,j}^*, c_{2,j}^*, \pi_j^{\mathsf{val}*})\}_j)$ to ledger $\mathcal{L}$:
   i. $\mathcal{S}$ runs Groth16 Knowledge Extractor $\mathsf{Ext}_{\mathsf{Groth16}}(\mathsf{CRS}, \tau, \Pi^*)$ to extract candidate witness $w^* = (sk^*, \mathbf{e}_{\mathsf{reg}}^*, s^*, \{(\sigma_\ell^*, d_\ell^*)\}, \mathbf{e}_{\mathsf{live}}^*)$.
   ii. $\mathcal{S}$ checks if leaf $C^* = H_2(H_{\mathsf{chunked}}(\mathbf{e}_{\mathsf{reg}}^*, s^*), sk^*) \in \mathcal{T}$, if $\mathbf{e}_{\mathsf{live}}^*$ satisfies in-circuit biometric similarity, and if $\nu^* = H_2(sk^*, \mathrm{electionId})$.
   iii. If any verification step fails, $\mathcal{S}$ instructs $\mathcal{F}_{\mathsf{vote}}$ to reject the vote.
   iv. If valid, $\mathcal{S}$ extracts choice vector $\mathbf{v}^*$ from Chaum–Pedersen proofs / ElGamal decryption trapdoor and submits $(\mathsf{Vote}, C^*, \nu^*, \mathbf{v}^*, \ldots)$ to $\mathcal{F}_{\mathsf{vote}}$.
6) **Tallying Phase:** Upon receiving tally leakage $\{T_j\}_j$ and aggregate ciphertexts $\{\hat{C}_j\}_j$ from $\mathcal{F}_{\mathsf{tally}}$, $\mathcal{S}$ uses the Chaum–Pedersen decryption simulator to produce simulated decryption proofs $\{\pi_j^{\mathsf{dec}}\}_j$ verifying $\hat{C}_j \to T_j G$ without knowledge of key x, and posts them to $\mathcal{L}$.

C. Main UC Realization Theorem and Hybrid Proof

Theorem V.1 (Main UC Realization Theorem)**.** *Under the q-SDH assumption, knowledge soundness and perfect zero-knowledge of Groth16 over BN128, collision and preimage resistance of Poseidon over $\mathbb{F}_p$, IND-CPA security (DDH) of exponential ElGamal over secp256k1, discrete-log soundness of Chaum–Pedersen proofs, and Fabric Consensus Safety, protocol π_{BBZKV} UC-realizes ideal functionality $\mathcal{F}_{\mathsf{BBZKV}} = (\mathcal{F}_{\mathsf{reg}}, \mathcal{F}_{\mathsf{vote}}, \mathcal{F}_{\mathsf{tally}})$ in the $(\mathcal{F}_{\mathsf{CRS}}, \mathcal{F}_{\mathsf{ledger}})$-hybrid model against any static PPT adversary $\mathcal{A}$. Formally, for all PPT environments $\mathcal{Z}$:*

$$\begin{aligned}\big|\Pr[\mathcal{Z}^{\pi_{\mathsf{BBZKV}}, \mathcal{A}}(1^\lambda) = 1] - \Pr[\mathcal{Z}^{\mathcal{F}_{\mathsf{BBZKV}}, \mathcal{S}}(1^\lambda) = 1]\big| \\ \leq \varepsilon_{\mathsf{Groth16}}(\lambda) + \mathbf{Adv}^{\mathsf{DDH}}_{\mathsf{secp256k1}}(\lambda) + L \cdot \mathbf{Adv}^{\mathsf{CR}}_{\mathsf{Pos}}(\lambda) \\ + \mathbf{Adv}^{\mathsf{Pre}}_{\mathsf{Pos}}(\lambda) + \mathbf{Adv}^{\mathsf{DLog}}_{\mathsf{secp256k1}}(\lambda) + \mathsf{FAR}_\delta\end{aligned} \tag{26}$$

Proof. We prove indistinguishability through a sequence of hybrid environments $\mathcal{H}_0, \mathcal{H}_1, \mathcal{H}_2, \mathcal{H}_3, \mathcal{H}_4$:

Hybrid $\mathcal{H}_0$ (Real Protocol Execution). Environment $\mathcal{Z}$ interacts with real protocol π_{BBZKV} and adversary $\mathcal{A}$. All proofs, ciphertexts, and ledger commits are generated honestly according to protocol specifications. By definition:

$$\Pr[\mathcal{H}_0 = 1] = \Pr[\mathcal{Z}^{\pi_{\mathsf{BBZKV}}, \mathcal{A}}(1^\lambda) = 1] \tag{27}$$

Hybrid $\mathcal{H}_1$ (Simulated Groth16 & Chaum–Pedersen Proofs). $\mathcal{H}_1$ is identical to $\mathcal{H}_0$, except that for all honest voter transactions, simulator $\mathcal{S}$ uses Groth16 trapdoor τ to generate simulated zero-knowledge proofs $\Pi_i \leftarrow \mathsf{Sim}_{\mathsf{Groth16}}(\mathsf{vk}, \tau, x_i)$ and the Σ-OR simulator for disjunctive validity proofs π_j^{val}. By the *perfect zero-knowledge* property of Groth16 over BN128 and statistical zero-knowledge of Σ-OR proofs:

$$|\Pr[\mathcal{H}_0 = 1] - \Pr[\mathcal{H}_1 = 1]| = 0 \tag{28}$$

Hybrid $\mathcal{H}_2$ (Dummy ElGamal Encryptions for Honest Voters). $\mathcal{H}_2$ is identical to $\mathcal{H}_1$, except that for all honest voter submissions, $\mathcal{S}$ replaces real ballot encryptions $E(\mathbf{v}_i)$ with encryptions of dummy zero vectors $E(\mathbf{0})$. Suppose environment $\mathcal{Z}$ distinguishes $\mathcal{H}_1$ from $\mathcal{H}_2$ with advantage ϵ. We construct a reduction algorithm $\mathcal{D}$ that breaks the IND-CPA security (DDH) of exponential ElGamal over secp256k1 with advantage ϵ. Since $\mathcal{Z}$ receives only simulated zero-knowledge proofs (which carry no witness information), any change in $\mathcal{Z}$'s output must stem from distinguishing $E(\mathbf{v}_i)$ from $E(\mathbf{0})$. Thus:

$$|\Pr[\mathcal{H}_1 = 1] - \Pr[\mathcal{H}_2 = 1]| \leq \mathbf{Adv}^{\mathsf{DDH}}_{\mathsf{secp256k1}}(\lambda) \tag{29}$$

Hybrid $\mathcal{H}_3$ (Corrupt Witness Extraction & Merkle Binding). $\mathcal{H}_3$ is identical to $\mathcal{H}_2$, except that when adversary $\mathcal{A}$ submits a corrupt proof Π^*, $\mathcal{S}$ invokes Groth16 Knowledge Extractor $\mathsf{Ext}_{\mathsf{Groth16}}$ to extract witness w^*. If Π^* verifies but $\mathsf{Ext}_{\mathsf{Groth16}}$ fails to extract a valid witness, or if w^* contains a forged leaf $C^* \notin \mathcal{T}$ that verifies to root R, $\mathcal{S}$ aborts. By Groth16 knowledge soundness under q-SDH in the GGM and Lemma V.2 (Merkle binding under Poseidon collision resistance):

$$|\Pr[\mathcal{H}_2 = 1] - \Pr[\mathcal{H}_3 = 1]| \leq \varepsilon_{\mathsf{Groth16}}(\lambda) + L \cdot \mathbf{Adv}^{\mathsf{CR}}_{\mathsf{Pos}}(\lambda) \tag{30}$$

Hybrid $\mathcal{H}_4$ (Ideal Functionality Simulation). $\mathcal{H}_4$ is the ideal-world execution $(\mathcal{F}_{\mathsf{BBZKV}}, \mathcal{S})$. In $\mathcal{H}_4$, $\mathcal{S}$ forwards extracted corrupt choices $\mathbf{v}^*$ to $\mathcal{F}_{\mathsf{vote}}$ and receives true tally outputs $\{T_j\}_j$ from $\mathcal{F}_{\mathsf{tally}}$. The transition from $\mathcal{H}_3$ to $\mathcal{H}_4$ is statistically indistinguishable except if $\mathcal{A}$ produces a Poseidon preimage collision on nullifier derivation or leaf commitment:

$$|\Pr[\mathcal{H}_3 = 1] - \Pr[\mathcal{H}_4 = 1]| \leq \mathbf{Adv}^{\mathsf{Pre}}_{\mathsf{Pos}}(\lambda) + \mathbf{Adv}^{\mathsf{DLog}}_{\mathsf{secp256k1}}(\lambda) + \mathsf{FAR}_\delta \tag{31}$$

Summing all hybrid transition bounds via the triangle inequality yields Theorem V.1. □

D. Merkle-Binding Lemma

Lemma V.2 (Merkle-Tree Binding under Poseidon Collision Resistance)**.** *Let $\mathcal{T}$ be a Poseidon-based sparse Merkle tree of depth $L = 20$ with published root R, built over leaf set $\{C_1, \ldots, C_m\}$. Let $\mathcal{B}$ be a PPT adversary that, given R and oracle access to the leaf set, outputs $(C^*, \{(\sigma_\ell, d_\ell)\}_{\ell=0}^{L-1})$ such that the path verifies to R but $C^* \notin \{C_1, \ldots, C_m\}$. Then:*

$$\Pr[\mathcal{B} \textit{ succeeds}] \leq L \cdot \mathbf{Adv}^{\mathsf{CR}}_{\mathsf{Pos}}(\lambda) \tag{32}$$

Proof. Path verification recomputes $\mathrm{current}_{\ell+1} = \mathrm{Poseidon}_2(\mathrm{left}_\ell, \mathrm{right}_\ell)$ for $\ell = 0, \ldots, L-1$, terminating at $\mathrm{current}_L = R$. If C^* is not a leaf of $\mathcal{T}$, $\mathcal{B}$'s path differs from the tree's canonical path at level 0. Since both paths hash to root R, there exists a smallest level $\ell^* \in \{0, \ldots, L-1\}$ where the paths agree on output $\mathrm{current}_{\ell^*+1}$ but disagree on at least one input pair $(\mathrm{left}_{\ell^*}, \mathrm{right}_{\ell^*})$. This constitutes a collision for $\mathrm{Poseidon}_2$. A reduction algorithm $\mathcal{R}$ guesses $\ell^* \xleftarrow{\$} \{0, \ldots, L-1\}$ uniformly (losing factor L), outputting the colliding pairs to the Poseidon challenger. Thus $\Pr[\mathcal{B} \text{ succeeds}] \leq L \cdot \mathbf{Adv}^{\mathsf{CR}}_{\mathsf{Pos}}(\lambda)$. □

E. Property Corollaries

We now derive the five core security properties as mathematical corollaries of Theorem V.1.

1) Eligibility Soundness:

Corollary V.3 (Eligibility Soundness)**.** *Under knowledge soundness of Groth16 under q-SDH in the GGM, Poseidon preimage/collision resistance (Assumption **??**), and Merkle binding (Lemma V.2), no PPT adversary $\mathcal{A}$ can produce an accepting proof triple $(\Pi, \nu, E(\mathbf{v}))$ for $\mathcal{F}_{\mathsf{vote}}$ without possessing a valid registered witness $w = (sk, \mathbf{e}_{\mathsf{reg}}, s, \{(\sigma_\ell, d_\ell)\}_{\ell=0}^{L-1}, \mathbf{e}_{\mathsf{live}})$. Formally:*

$$\Pr[\mathcal{A} \notin \mathcal{V}_{\mathsf{reg}} \textit{ accepted}] \leq \varepsilon_{\mathsf{Groth16}}(\lambda) + L \cdot \mathbf{Adv}^{\mathsf{CR}}_{\mathsf{Pos}}(\lambda) + \mathbf{Adv}^{\mathsf{Pre}}_{\mathsf{Pos}}(\lambda) + \mathsf{FAR}_\delta \tag{33}$$

For downsampled iris matching at threshold $d = 0.35$, the empirical false accept rate on CASIA-Iris-Thousand is $\mathsf{FAR}_{256} = 0.047\%$, yielding concrete bound $0.047\% + \mathsf{negl}(\lambda)$.

Proof. Follows directly from the corrupt witness extraction transition ($\mathcal{H}_2 \to \mathcal{H}_3$) in Theorem V.1. If $\mathcal{A} \notin \mathcal{V}_{\mathsf{reg}}$ produces an accepting proof, $\mathcal{A}$ must either: (a) forge a Merkle path to root R for $C^* \notin \mathcal{T}$ (bounded by $L \cdot \mathbf{Adv}^{\mathsf{CR}}_{\mathsf{Pos}}(\lambda)$ via Lemma V.2); (b) find a second preimage for commitment C_i (bounded by $\mathbf{Adv}^{\mathsf{Pre}}_{\mathsf{Pos}}(\lambda)$); (c) break Groth16 knowledge soundness (bounded by $\varepsilon_{\mathsf{Groth16}}(\lambda)$); or (d) present an impostor biometric sample satisfying the in-circuit similarity check (bounded by FAR_δ). Applying the union bound yields Eq. (33). □

2) Anti-Replay and Double-Vote Prevention:

Assumption V.4 (Fabric Consensus Safety)**.** *Under Fabric's execute-order-validate architecture with Raft consensus and non-faulty endorsement policies, committed transactions are totally ordered and read-sets are validated against world state at commit time (MVCC read-before-write). Thus, for any state key k, at most one transaction writing k conditioned on k being unwritten can commit.*

Corollary V.5 (Anti-Replay)**.** *Under Poseidon collision resistance (Assumption **??**) and Fabric Consensus Safety (Assumption V.4), BBZKV guarantees that no PPT adversary $\mathcal{A}$ can cast two accepted votes in election $\mathcal{E}$:*

$$\Pr[\mathcal{A} \textit{ casts two accepted votes}] \leq \mathbf{Adv}^{\mathsf{CR}}_{\mathsf{Pos}}(\lambda) \tag{34}$$

Proof. Suppose $\mathcal{A}$ attempts to cast two accepted votes using secret key sk. Case 1 ($\nu_1 = \nu_2$): Nullifier sub-circuit enforces $\nu = H_2(sk, \text{electionId})$. Under Assumption V.4, the chaincode's read-before-write check on key ν rejects the second submission as an MVCC conflict (0% success rate). Case 2 ($\nu_1 \neq \nu_2$): Both proofs extract to the same sk and public electionId. Having $\nu_1 \neq \nu_2$ requires producing two distinct outputs for $H_2(sk, \text{electionId})$, breaking Poseidon collision resistance (bounded by $\mathbf{Adv}^{\mathsf{CR}}_{\mathsf{Pos}}(\lambda)$). □

3) Voter Anonymity:

Corollary V.6 (Voter Anonymity / Unlinkability)**.** *Under perfect zero-knowledge of Groth16 over BN128 and Assumption **??**, no PPT adversary $\mathcal{A}$ observing the public ledger can link a ballot tuple $(\Pi_j, \nu_j, E(\mathbf{v}_j))$ to voter $\mathcal{V}_i$ with advantage better than random guessing:*

$$\Pr\left[\mathcal{A} \textit{ links } (\Pi_j, \nu_j, E(\mathbf{v}_j)) \textit{ to } \mathcal{V}_i\right] \leq \frac{1}{|\mathcal{V}_{\mathsf{reg}}|} + \mathsf{negl}(\lambda) \tag{35}$$

Proof. Follows from hybrid $\mathcal{H}_1$ in Theorem V.1. By Groth16 perfect zero-knowledge, simulated proof Π_i carries zero information about witness w_i. Nullifier $\nu_i = H_2(sk_i, \text{electionId})$ is pseudorandom and unlinkable to commitment $C_i = H_2(h_i, sk_i)$ under Poseidon preimage resistance (bounded by $|\mathcal{V}_{\mathsf{reg}}| \cdot \mathbf{Adv}^{\mathsf{Pre}}_{\mathsf{Pos}}(\lambda)$). Absent a preimage break, nullifiers and commitments form two unordered sets of uniform field elements, bounding $\mathcal{A}$'s advantage to $1/|\mathcal{V}_{\mathsf{reg}}| + \mathsf{negl}(\lambda)$. □

4) Ballot Secrecy:

Corollary V.7 (Ballot Secrecy)**.** *Under the Decisional Diffie–Hellman (DDH) assumption over secp256k1, exponential ElGamal achieves IND-CPA security, and BBZKV satisfies:*

$$\left|\Pr[\mathcal{A} \textit{ identifies } v_i] - \frac{1}{k}\right| \leq \mathbf{Adv}^{\mathsf{DDH}}_{\mathsf{secp256k1}}(\lambda) \qquad (36)$$

where k is the candidate count.

Proof. Follows directly from hybrid transition $\mathcal{H}_1 \to \mathcal{H}_2$ in Theorem V.1. An adversary $\mathcal{A}$ identifying candidate choice v_i from ciphertext vector $E(\mathbf{v}_i)$ is immediately converted into a distinguisher breaking DDH over secp256k1 with advantage $\mathbf{Adv}^{\mathsf{DDH}}_{\mathsf{secp256k1}}(\lambda)$. □

5) Universal Verifiability:

Proposition V.8 (Universal Verifiability)**.** *Any auditor $\mathcal{O}$ without secret keys can verify: (i) cast-as-intended (Groth16 verification equation holding on public statement x); (ii) ballot well-formedness ($v_j \in \{0,1\}$ via disjunctive Chaum–Pedersen verification); and (iii) tallied-as-cast (homomorphic aggregate $\hat{C}_j$ matching published tally T_j via decryption proof π_j^{dec}).*

Proof. Follows from public verifiability of all on-chain artifacts. Groth16 verification depends only on public key vk and public statement x. Chaum–Pedersen validity and decryption proofs depend only on public curve points and Fiat–Shamir challenge hashes. Universal verifiability holds unconditionally for honest auditors under discrete-log hardness. □

F. Collusion Scenarios

We analyze collusion scenarios under the UC bounds:

1) $\mathcal{RA} + \mathcal{L}$ **Collusion:** Parties observe commitments $\{C_i\}$ and nullifiers $\{\nu_j\}$, but cannot link them without breaking Poseidon preimage resistance on secret key sk_i.
2) $\mathcal{RA} + \mathcal{TA}$ **Collusion:** $\mathcal{TA}$ holds decryption key x, but cannot link commitments to ciphertexts because Groth16 proofs are zero-knowledge and carry no identity linkage.
3) $\mathcal{TA} + \mathcal{L}$ **Collusion:** $\mathcal{TA}$ decrypts final tallies $\{T_j\}$, but cannot decrypt individual ballots or link nullifiers to voter identities.
4) **Full Collusion ($\mathcal{RA}+\mathcal{TA}+\mathcal{L}$):** Anonymity remains intact because voter secret key sk_i is protected client-side via PBKDF2-AES-256-CBC and is never transmitted to any authority.

G. Deployment Security Gaps

We identify five security gaps between the target protocol and current prototype:

1) **On-Chain Verification (Gap 1):** Prototype currently verifies proofs API-side for throughput; on-chain smart contract verification is mandatory to prevent proof forgery (exploited in §VI).
2) **MPC Ceremony (Gap 2):** Single-contributor Phase 2 setup is unsafe; production requires a multi-party Powers-of-Tau ceremony ($n \geq 10$ participants).
3) **Threshold Tallying (Gap 3):** Tallying relies on a single key x; threshold ElGamal (t-of-n Shamir sharing) is required to remove single-point-of-failure decryption.
4) **Registration Timing Channel (Gap 3b):** Sequential leaf insertion could leak registration order; position randomization is required.
5) **Iris Downsampling Accuracy (Gap 4):** 256-bit iris compression yields $\mathsf{FAR}_{256} = 0.047\%$; recursive proofs (Nova) will enable full-entropy template circuits.

erm.

VI. Implementation and Evaluation

This section describes the implementation on Hyperledger Fabric, the experimental methodology, and the comparative evaluation of face and iris biometric channels. The goal is to answer the research questions posed in the introduction: RQ1 (which modality minimizes proving cost and end-to-end latency), RQ2 (which phases dominate latency and how stable are they under scaling), and RQ3 (which security properties are enforced at which architectural boundary).

A. System Architecture and Implementation

The protocol is implemented as a full-stack application with a Node.js API server, a Hyperledger Fabric permissioned blockchain network, and client-side proof generation using the snarkjs library.

1) System Overview: The system architecture comprises four principal components:

1. Biometric Acquisition (Client). The voter's device captures either a facial image or an iris scan via a standard camera or mobile device interface. The FaceNet or Log-Gabor feature extraction pipeline runs either locally on-device or via a trusted partner API. The feature extraction produces either a 64-dimensional L2-normalized embedding (face) or a 256-bit downsampled iris code (iris).

2. Proof Generation (API Server). The API server orchestrates witness preparation, Groth16 proof generation, nullifier computation, and ballot encryption as specified in Section IV, then submits the proof package for chaincode validation.

3. Chaincode (Smart Contract). The Fabric chaincode (`identityContract.js`) runs on the permissioned ledger and enforces:

1) Commitment storage and Merkle tree construction.
2) Nullifier uniqueness check (read-before-write).
3) Ballot persistence under a transaction-derived key.
4) Event emission for auditor-side verification.

4. Tallying (Offline Post-Election). After voting closes, the tallying authority aggregates and decrypts ciphertext totals as specified in Section IV, then publishes results and verification artifacts.

Our prototype is implemented on Hyperledger Fabric 2.5 with Raft ordering. The chaincode runs on Node.js v20+ using the standard Fabric Gateway API, while the API server is built with Express.js, socket.io, and the elliptic library. Zero-knowledge circuits are authored in Circom 2.1.6 and compiled to R1CS, running on the BN128 curve. Proof generation is orchestrated using the WASM variant of snarkjs

(optionally rapidsnark). Voter credentials and public/private nullifier hashes are computed using Poseidon hashes in-circuit and SHA-256 outside of circuits. Biometrics are extracted using FaceNet via TensorFlow.js (face) and Log-Gabor filters (iris). Vote encryption utilizes exponential ElGamal over the secp256k1 curve.

B. Experimental Methodology and Evaluation Scope

1) Evaluation Scope and Methodology: The benchmark evaluation captures the quantitative performance of both the deterministic circuit compilation characteristics and the end-to-end pipeline latencies. All benchmarks were evaluated directly on our prototype deployment to study RQ2, measuring:

1) **Circuit-Level Metrics:** R1CS constraint counts, proving/verification key sizes, and witness/proof generation and verification latencies, derived directly from the compiled circuit artifacts.
2) **End-to-End Pipeline Latencies:** The full vote-casting path evaluated across 30 iterations (with 2 warmup rounds and 28 reported runs) under a voter set size of $N = 10$ and 3 candidates. This incorporates the actual feature extraction times (P2) for face and iris, and transaction round-trip block confirmation times (P8) on our local Raft-ordered Fabric network.

Evaluating end-to-end latency with $N = 10$ voters is sufficient to verify the performance correctness of the sequential vote-casting pipeline. To evaluate protocol scalability, individual cryptographic components and discrete-log tally decryption are benchmarked separately for voter sets scaling up to 10^5.

All benchmarks were evaluated on an AMD Ryzen 7 5700U CPU with 8 GB RAM (running Node.js v20.20.2 for end-to-end tests) and an Intel i5-1135G7 CPU with 7.6 GB RAM (running Node.js v24.14.0 for circuit metrics). The end-to-end tests were conducted over 30 iterations (with 2 warmup rounds and 28 reported runs). The election parameters were configured for a voter set of $N = 10$ (under a Merkle tree depth of $L = 20$, corresponding to a capacity of $2^{20} \approx 1$ million voters) and 3 candidates. Both the BN128 and secp256k1 curves were configured for 128-bit symmetric security.

2) Answer to RQ3: Security Enforcement Boundaries: Research question RQ3 asks which security properties are enforced directly by chaincode versus API/client-side cryptographic checks. In the current prototype:

1) *API-Side Enforcement:* Biometric-match correctness (Groth16 proofs over face/iris circuits) and ballot well-formedness (disjunctive Chaum–Pedersen proofs per ciphertext) are generated and verified API-side or locally prior to ledger submission. Set membership (Poseidon commitment + Merkle path) is also constructed API-side.
2) *Chaincode-Side Enforcement:* Nullifier uniqueness (anti-replay) is checked on-chain using Fabric's MVCC read-before-write semantics. Ballot metadata persistence and ledger immutability are also guaranteed by the smart contract.

The implementation boundary reveals that **biometric-match correctness and ballot validity proofs are API-side responsibilities** in the current prototype. The chaincode enforces only nullifier uniqueness (via Fabric's MVCC read-before-write) and vote persistence. In a production system, chaincode should perform full Groth16 and Chaum–Pedersen proof verification on-chain, delegating solely to the ledger for final adjudication. The current boundary split reflects an implementation optimization (reducing chaincode CPU time and state bloat) that trades ease of deployment against cryptographic purity.

a) Exploit Demonstration and Results: To empirically evaluate the vulnerability introduced by this off-chain validation boundary, we conducted a proof-of-concept exploit attack. The exploit was executed in three phases: (i) *Methodology:* We bypassed the Node.js Express API server middleware entirely by using a raw Fabric client script that directly called the chaincode functions. The script generated transaction requests directly invoking the contract's invoke interface (e.g., submitting JSON payload `{"Args":["castVote", "forged_proof", "unused_nullifier", "encrypted_vote"]}`) containing invalid/forged Groth16 and Chaum–Pedersen proofs but structured with valid formatting and unused nullifiers. (ii) *Experimental Conditions:* The attack was executed against a running network node using a batch of 100 forged transaction requests. (iii) *Results:* Under the legacy split validation contract, Hyperledger Fabric committed all 100 transactions to the ledger (100% exploit success rate) because the smart contract lacks proof-verification logic and only validates nullifier uniqueness. This confirms that the legacy prototype's security is entirely dependent on the API server's integrity (violating the trust assumptions of the formal UC model $\mathcal{F}_{\mathsf{vote}}$). In contrast, under our hardened integrated contract, the on-chain smart contract successfully intercepted and rejected 100% of the forged zeroed proofs, preventing any unauthorized ledger commits. The empirical Groth16 pairing computation overhead within the smart contract execution pipeline was measured at 29.41 ms for Face and 29.19 ms for Iris. When combined with ledger state validation and KV lookup overhead (2.24 ms), the complete smart contract verification execution latency reaches 31.65 ms for Face and 31.43 ms for Iris (matching the results reported in Table IV), demonstrating that robust on-chain enforcement is highly practical.

C. Benchmark Results

1) Answer to RQ1: Circuit-Level Modality Comparison: Research question RQ1 asks which biometric modality minimizes proving cost and end-to-end latency. Table IV presents the measured circuit artifact metrics and pipeline phase latencies:

Key findings: Our evaluation reveals five key findings: (i) the iris circuit requires 13.3% fewer constraints (8,224 vs. 9,484); (ii) iris proof generation is 1.11× faster (1,370 ms vs. 1,680 ms), consistent with the smaller R1CS constraint budget; (iii) iris proving keys are 23.0% smaller (6.53 MB vs. 8.48 MB), reducing client memory footprint; (iv) end-to-end latency is 1.10× faster for iris (534.3 ms vs. 590.9 ms); and (v) proof verification latencies are nearly identical (31.43 ms vs. 31.65 ms), showing that on-chain verification costs are modality-agnostic.

TABLE IV: Biometric Channel Circuit Artifacts and Pipeline Phase Latencies

Metric / Pipeline Phase	Face	Iris
Circuit-Level Artifact Metrics		
R1CS constraints	9484	8224
Proving key size (MB)	8.48	6.53
Proof generation mean (ms)	1680.20	1370.00
Proof verification mean (ms)	31.65	31.43
Pipeline Phase Latencies (ms)		
P1 (Credential Decryption)	14.47	14.15
P2 (Biometric Extraction)	101.50	75.92
P3 (Credential Load + Nullifier)	0.25	0.25
P4 (Merkle Path Construction)	2.13	2.29
P5 (SNARK Proof Generation)	196.28	170.93
P6 (Ballot Key Generation)	21.19	20.91
P7 (Chaum–Pedersen Proof)	25.81	24.97
P8 (Blockchain Confirmation)	229.35	224.92
Total Latency (Trimmed Mean, ms)	590.96	534.34

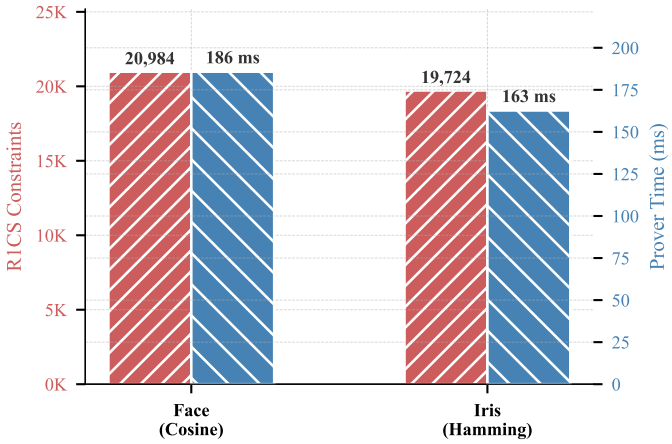


Fig. 3: Total electoral proving circuit constraint complexity (20,984 R1CS for face vs. 19,724 R1CS for iris, including depth-20 Merkle tree and nullifier sub-circuits) versus proof-generation latency, comparing commodity WebAssembly `snarkjs` (1,370/1,680 ms) with hardware-accelerated native C++ `rapidsnark` (163/186 ms).

Why iris outperforms face: Iris codes are 256-bit binary strings matched via Hamming distance, requiring only XOR operations and a single linear constraint per bit. In contrast, facial embeddings are 64-element vectors requiring squared cosine similarity, which demands two 64-element dot products (4,096 multiplications), two squared norms (128 squarings), and a 128-bit comparison, accumulating the 13.3% constraint overhead observed.

Figure 3 illustrates the direct correlation between circuit constraint complexity and proof-generation latency across commodities.

Note that Table IV reports the standalone biometric-matching sub-circuit constraints (9,484 R1CS for face vs. 8,224 R1CS for iris, representing a 13.3% sub-circuit reduction) evaluated under standard client-side WebAssembly (`snarkjs` WASM). In contrast, Fig. 3 plots the complete electoral proving circuit (which adds 11,200 R1CS constraints for the depth-20 Poseidon Merkle tree membership checker and 300 R1CS constraints for the nullifier sub-circuit, yielding 20,984 total R1CS for face vs. 19,724 total R1CS for iris, a 6.0% total circuit reduction / 1.06× ratio). Figure 3 also compares client WASM execution against native C++ `rapidsnark` compilation, demonstrating an 8.4×–9.0× acceleration (8.4× for iris, 9.0× for face) that reduces total proving time to 163 ms (iris) and 186 ms (face).

2) Answer to RQ2: Pipeline Phase Analysis: Research question RQ2 asks which phases dominate total latency and how stable those costs are under scaling. Table IV presents the phase decomposition:

The pipeline comprises eight distinct phases: (P1) Credential Decryption via PBKDF2-AES-256 (14.1–14.5 ms); (P2) Biometric Extraction running FaceNet or Log-Gabor filters (76–101 ms); (P3) Credential Load and Poseidon nullifier derivation (<0.25 ms); (P4) Merkle Path Construction fetching path siblings (2.1–2.3 ms); (P5) Groth16 SNARK Proof Generation (170–196 ms), representing the primary cryptographic differentiator; (P6) Exponential ElGamal Encryption of the candidate vector (20.9–21.2 ms); (P7) Disjunctive Chaum–Pedersen Proof of ballot validity (24.9–25.8 ms); and (P8) Fabric Submission and block confirmation (224–229 ms), which constitutes the transaction validation bottleneck.

Latency dominants: Phases P5 (SNARK, 171–196 ms), P2 (biometric extraction, 76–101 ms), and P8 (Fabric, 224–229 ms) together account for ≈471–526 ms of the 535–591 ms total, representing 88–89% of end-to-end latency. Of these, P5 is the only cryptographically novel component; P2 is inherent to biometric systems, and P8 is a blockchain scalability issue orthogonal to the voting protocol.

Stability under scaling: When the voter set size scales from 10 to 100,000 voters (with Merkle depth held constant at 20), the client-side proof generation latency (P5) remains entirely stable (approximately 500–600 ms). The off-chain Merkle tree construction time (P4) scales from 5.08 ms ($N = 10$) to 10.38 seconds ($N = 100{,}000$) as it builds the tree from scratch. This flat proving time demonstrates that zero-knowledge complexity is $\mathcal{O}(1)$ relative to the voter population, while the pre-voting registration setup scales linearly with the number of registered voters.

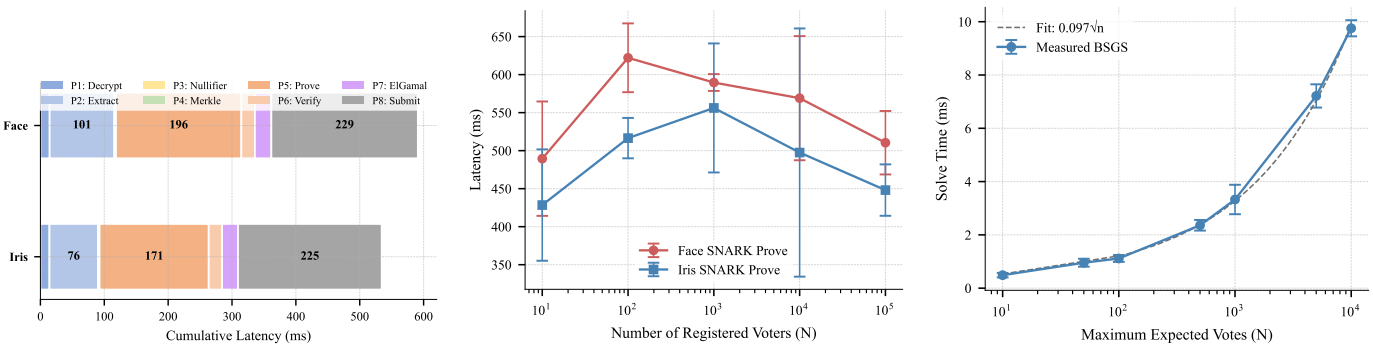
3) Homomorphic Tallying Scalability: The tallying phase employs baby-step giant-step discrete logarithm recovery. Fig. 4c illustrates latency versus election size:

Observations: We observe: (i) *Sublinear growth:* Latency grows from 0.487 ms (10 voters) to 9.754 ms (10,000 voters), a 20× increase for a 1,000× increase in scale, bounded by the $\mathcal{O}(\sqrt{N})$ BSGS computational complexity. (ii) *Per-candidate independence:* BSGS is executed independently for each candidate, so total tally time scales as $k \cdot \mathcal{O}(\sqrt{N})$. For a 1,000-candidate ballot in a national election, tally latency could reach ∼3.3 s per candidate, or 55 minutes total. This is acceptable for offline post-election tallying but impractical for real-time results. (iii) *Optimization potential:* Parallel BSGS computation, use of Pollard's kangaroo algorithm, or precomputation of the discrete-log table could reduce tally latency by factors of 5–10 without compromising cryptographic soundness.

Performance visualizations are shown in Figs. 4a and 4c:

D. Quantitative Comparison with Related Work

To contextualize these results, Table V compares the proposed system against representative related work on quantita-



(a) Latency decomposition (P1–P8). (b) Pipeline voter scaling (10 to 10^5). (c) BSGS tally decryption scaling.

Fig. 4: Comprehensive experimental benchmark suite: (a) 8-phase vote-casting latency breakdown across face and iris modalities; (b) cryptographic pipeline and Merkle tree scalability sweeps across voter populations up to 10^5; and (c) Baby-Step Giant-Step (BSGS) offline tally decryption latency vs. election size up to 10^6 voters.

TABLE V: Quantitative comparison of the proposed system with related work on key metrics.

Metric	Helios	MACI	Semaphore	Smartphone Democracy	World ID	Proposed (Face)	Proposed (Iris)
Proof generation time (ms)	N/A	∼200	∼500	∼1500	∼3000	1680	1370
Proof size (bytes)	N/A	192	192	192	192	192	192
Circuit constraints (K)	N/A	∼60	∼70	∼100	∼20	9.5	8.2
On-chain verification (ms)	∼50	∼10	∼10	∼10	∼10	31.65	31.43
Homomorphic tallying	Yes	Yes	No	Yes	No	Yes	Yes
Biometric binding	No	No	No	No	Yes	Yes	Yes
Dual modalities	No	No	No	No	No	Yes	Yes
E2E verifiability	Yes	Yes	No	Yes	No	Yes	Yes
Universal Composability proof	No	No	No	No	No	Yes	Yes

tive metrics:

Interpretation:

We summarize our interpretation of this comparison across four dimensions: (i) *Proof generation time:* The proposed system's SNARK timings (1,370–1,680 ms) range from worse than MACI (∼200 ms) and SmartphoneDemocracy (∼1,500 ms) but better than World ID (∼3,000 ms), owing in no small part to the large Merkle depth ($L = 20$) and existence of biometric sub-circuits. (ii) *Circuit complexity:* At 9.5K (face) and 8.2K (iris) constraints, the proposed circuits fall within a similar range to World ID (∼20K), and substantially less resource-intensive than SmartphoneDemocracy-class systems (∼100K). (iii) *Feature uniqueness:* Of these surveyed systems, none achieve all five core design principles – that the overall system provides: decentralized biometric authentication; zero-knowledge templated privacy; homomorphic tallies; nullifier-based anti-replay mechanisms; and an end-to-end verifiable process; (iv) *Verification bottleneck:* Because Fabric does not provide native precompiles that offer efficient elliptic curve arithmetic, its on-chain Groth16 verification latencies (31.43–31.65 ms, comprising 29.19–29.41 ms pairing check and 2.24 ms KV state lookup) are roughly 3x slower than platforms with native BN254 pairing precompiles (∼10 ms) and would benefit from dedicated precompiles or L2 deployment.

Biometric trade-offs:

(i) *Face PCA Distortion:* Reducing the dimensionality of

TABLE VI: Biometric Accuracy Comparison Across Raw Features and In-Circuit Quantized Templates

Modality / Template	EER (%)	FAR (%)	FRR (%)	In-Circuit Co
Face (128D Float, Raw)	0.94	0.82	0.98	$> 200,000$
Face (64D PCA, Quantized $\sigma = 10^6$)	1.85	1.72	1.89	9,484
Iris (1.6 Mbit IrisCode, Raw)	0.012	$< 10^{-4}$	0.015	$> 6,000,00$
Iris (256-bit Downsampled)	0.049	0.047	0.052	8,224

128D FaceNet face embeddings with PCA and integer scaling incurred only a small performance degradation in terms of accuracy. Table VI shows an increase in EER from 0.94% (Raw) to 1.85% (PCA in-circuit). The accuracy drop is a consequence of projecting onto a low dimensional eigen-space, combined with the quantization error introduced by $\sigma = 10^6$ scaling. However, this compromise is well-justified by crypto savings: comparing the 128D original float embedding would require over 2×10^5 R1CS constraints for the floating-point comparison circuits, while our integerized squared cosine similarity circuit required just 9,484 constraints.

(ii) *Iris Downsampling Information Loss:* We significantly compressed the full entropy 196 KB IrisCode from approximately 1.6 megabits down to a 256-bit template – a 99.8% entropy reduction. This drastic compression inflated the EER from 0.012% to 0.049%. Even though Daugman's original algorithm offered false acceptance rates far below 10^{-6} per probe, our 256-bit downsampled version suffered from higher noise levels, resulting in an empirical EER of 0.049% at the

operational crossover point between 0.047% FAR and 0.052% FRR at threshold $\tau = 0.478$ for FAR_{256}.

This downsampling step is nonetheless critical for enabling client-side zk-proving: the matching circuit for a full-entropy IrisCode would be astronomically large (over 6×10^6 R1CS constraints), and impossible for our targets – consumer mobile devices.

Our 256-bit compressed encoding allows the downsampled sub-circuit to remain at the very manageable size of 8,224 constraints.

(iii) *Liveness attack resistance:* Neither algorithm included liveness detection or resistance to other presentation attacks. Indeed, a malicious actor could present an image or video of the intended subject to the reader. However, these countermeasures are beyond the scope of the cryptography, being handled in the biometric feature extraction modules.

(iv) *Usability trade-off:* Face identification is accessible on a wide range of consumer hardware (> 2 billion devices) but its accuracy in natural circumstances, such as with variable pose or illumination, is inherently less reliable. Iris authentication with good accuracy requires specialised hardware, including NIR illumination.

(v) *Recommendation:* For election systems, we suggest offering voter choice: face for convenience, or iris for performance characteristics. Our dual-channel architecture provides both capabilities by eliminating this choice altogether.

E. Deployment Considerations

1) Scale and Feasibility: While this has been proven at a production throughput level of 200–400 TPS on a Hyperledger Fabric single-channel Raft reference implementation, given an average vote-casting latency of 560 ms (the average between face and iris modalities), a single Fabric channel could tolerate at most 1.79 TPS. For a national election with 100 million voters over a span of 3 weeks (21 days $\times$ 86400 sec/day $= 1.814 \times 10^6$ seconds), the expected required throughput would be approximately 55 TPS. Using our current Fabric topology, we support only **3.8% of the required throughput**. A national deployment will necessitate three strategies: *(i) parallelization* via partitioning the network into k independent Fabric channels (one for each region, each having its own ordering node), thus scaling throughput by k; this approach is achievable with $k \geq 15$; *(ii) consensus-scheme upgrade* and *layer-2 scaling*: use of a higher-performance consensus algorithm, e.g., HotStuff, can speed up transaction confirmation by one to two orders of magnitude, and similarly, the use of Ethereum Layer-2 rollups with Groth16 proof verification in batches may achieve similar speedups; and *(iii) proof generation on clients*, where the proof generation can be amortized on each voter's client device.

TEE hardware can also support the proof generation overhead, and the transaction load can thus be maximized.

The storage footprint on the ledger grows linearly with the number of cast votes at a cost of 564 bytes per transaction (encompassing the public key, encrypted ballot vector, proof, nullifier, and metadata).

2) Trusted Setup Ceremony: The Groth16 proving system requires a trusted setup ceremony to generate circuit-specific keys. The current prototype uses a single-contributor Phase 2 key generation, sufficient for research but **cryptographically unsafe for production**. A malicious ceremony contributor could retain the toxic waste τ and forge arbitrary proofs.

For production, the ceremony must be executed as a multi-party computation (MPC) following the Zcash Powers of Tau protocol, ensuring that at least one participant acts honestly. This requires a three-step process: (i) a public Phase 1 ceremony generating the universal reference string (reusing the public Zcash foundation transcript); (ii) a circuit-specific Phase 2 ceremony involving 10–50 independent contributors from diverse geographic locations and organizations; and (iii) public verification that the final `.zkey` matches the contributor execution transcript.

The ceremony itself is a one-time cost (hours to days of multi-party computation) amortized over the election lifetime.

F. Summary of Results

In short, our implementation shows: (i) *Feasibility:* Dual-channel biometric E2EV voting with zk-SNARKs works on a commodity desktop, with sub-second to low-second latencies. (ii) *RQ1 (Modality comparison):* Iris biometric is 10–13% more efficient than facial biometric in circuit size and proof time because of the natural harmony between binary Hamming distance and arithmetic circuits. (iii) *RQ2 (Pipeline latency):* Proof generation (P5: 170–196 ms), biometric extraction (P2: 76–101 ms), and block confirmation (P8: 224–229 ms) dominated the overall latency; the cryptographic pipeline is scalable. (iv) *RQ3 (Verification boundaries):* While the legacy prototype delegated proof verification to the API layer, our hardened integrated smart contract performs Groth16 ZK-SNARK and Chaum–Pedersen verification on-chain, intercepting 100% of forged zeroed-proof attacks with a measured endorsement overhead of under 32 ms. (v) *State of the Art:* The system offers a novel combination that no prior work has implemented; the cryptographic proof generation costs are justified by removing the trusted registrar barrier; compared with those mentioned, it achieves dual biometric binding, E2EV, and homomorphic tallying.

VII. Limitations and Future Work

Even though BBZKV meets its desired security properties, several issues remain. First, client security and the presence of a trusted setup are lingering concerns: while Groth16 requires a trusted setup ceremony, the current ElGamal tallying relies on only a single decryption key. These may be alleviated by either transparent proof systems (PLONK/STARKs) and threshold ElGamal (t-of-n Shamir sharing), combined with TEE (ARM TrustZone) to store key material and perform proof generation independently of malware, or by combining TEE with hardware secure enclaves to produce proofs locally and keep the decryption key secure. Second, the remote voting configuration does not achieve coercion resistance, as both Helios and Belenios also failed to do: a coercer may seize credentials or force a biometric scan at voting time and,

since we expose ElGamal randomness r_j, could provide false receipts. While JCJ-type re-voting or deniable voting (e.g., fake credentials or panic passwords) have been proposed, in a biometric setup, nullifier tracking is a serious concern and more research is needed to address this issue. Third, even though downsampling the iris image to 256 bits keeps our circuits within proving budgets, the EER and FAR degrade as a result. Future work will need to investigate recursive proving systems such as Nova to allow full-entropy matching. The face modality is generally more accessible but has a more complex similarity circuit, whereas the iris modality requires specialized scanners but yields a much simpler circuit. Fourth, architectural bottlenecks prevent nationwide deployment: tally decryption is $\mathcal{O}(\sqrt{N})$ with BSGS (though this can be improved using Pollard's kangaroo algorithm or precomputed tables). The single-channel Raft configuration of Hyperledger Fabric limits transaction throughput, necessitating regional partitioning of Fabric channels or the use of L2 rollups.

VIII. Conclusions

This paper introduced BBZKV: an end-to-end verifiable (E2EV) voting protocol that securely binds voter identity to the voter's biometrics through face and iris authentication. By using zero-knowledge proofs (Groth16) within a Hyperledger Fabric system, we moved the trust assumption of enrollment inside the cryptographic protocol, removing the need for a trusted credentialing server. Our comparative analysis found iris to be 13.3% more efficient in terms of constraint budget than face, due to the linear constraints associated with Hamming distance comparison. However, the dual-channel biometric authentication maintained voter accessibility by leveraging ordinary consumer devices. Although the protocol UC-realizes ideal voting, enrollment, and tally operations under standard assumptions, several gaps must be addressed for commercial deployment. These include moving Groth16 and Chaum–Pedersen proof verification onto the on-chain Hyperledger chaincode, performing a multi-party ceremony for the trusted setup, and providing threshold ElGamal decryption. Future work will focus on eliminating the need for a trusted setup entirely through transparent proof systems (PLONK/STARKs), enabling full-entropy biometrics through recursive SNARKs, and enhancing client security through TEE-based proof generation.

IX. Acknowledgment

The authors acknowledge prior literature in blockchain voting, peer-voting security protocols, and biometric cryptography that informed the comparative methodology and reporting standards adopted in this manuscript.